MLP Viva: Unique Questions & Answers

Compiled from student experiences across Level 1, Level 2, and Level 3 vivas

---

NOTEBOOK & PROJECT EXPLANATION

Q1. How should I present my notebook?

Walk through your notebook section by section: problem statement → EDA → preprocessing (imputation, encoding, scaling) → feature engineering → model training → hyperparameter tuning → model comparison → insights. Explain WHY you did each step, not just WHAT. Take 20–30 minutes. A thorough explanation leaves less room for additional questions.

Q2. What is the objective/problem statement of your project?

Clearly state what you are predicting (target variable), whether it is a classification or regression problem, and why it matters. E.g., 'We predict X using Y features; this is a binary classification problem because the target has two classes.'

Q3. What insights did you gain from EDA?

Mention at least 3–5 concrete findings: class imbalance ratios, distribution shapes (skewed/normal), key correlations, outlier presence, missing value patterns, and any feature relationships that informed your preprocessing or feature engineering choices.

Q4. What is your best model and why does it perform better?

Name your best model, give its score, and compare it against baseline/other models. Explain why it performs better — e.g., tree-based models handle non-linearity well, boosting corrects previous errors, etc. Reference train vs. validation scores to show it is not overfitting.

EDA & VISUALISATION

Q1. What is Exploratory Data Analysis (EDA) and why do we do it?

EDA is the process of visually and statistically summarising a dataset before modelling. We do it to understand distributions, detect missing values and outliers, find correlations between features, check class balance, and form hypotheses about which features matter most.

Q2. What is a correlation matrix and how do you read it?

A correlation matrix shows pairwise Pearson (or Spearman) correlation coefficients between numeric features, ranging from –1 (perfect negative) to +1 (perfect positive). Values near 0 mean no linear relationship. You use it to drop highly correlated redundant features and to identify predictors strongly related to the target.

Q3. What is a box plot (box-and-whisker plot)?

A box plot displays a feature's quartiles: the box spans Q1 to Q3 (IQR), the line inside is the median, and the whiskers extend to 1.5×IQR. Points beyond the whiskers are plotted as outliers. It quickly shows spread, skew, and outlier presence.

Q4. What is univariate and bivariate analysis?

Univariate analysis studies one variable at a time (histograms, box plots, descriptive stats). Bivariate analysis studies the relationship between two variables (scatter plots, correlation, cross-tabulations). Together they reveal individual distributions and inter-feature relationships.

Q5. What does a histogram tell you?

A histogram shows the frequency distribution of a numeric variable. You can infer whether the distribution is normal, skewed (left/right), bimodal, or uniform, and spot outliers or data-entry errors as isolated bars far from the main mass.

Q6. How can you find correlation between categorical features?

Use Chi-Square test of independence, Cramér's V, or convert categories to dummy variables and compute point-biserial correlation with the target. For ordinal categories, Spearman correlation is appropriate.

DATA PREPROCESSING

Q1. What is data preprocessing and why is it needed?

Preprocessing transforms raw data into a form suitable for ML models. It includes handling missing values, removing/treating outliers, encoding categoricals, scaling numerics, and splitting data. Without it, models may produce biased or inaccurate results because most algorithms cannot handle missing data, mixed types, or vastly different feature scales.

Q2. What is imputation and what are its techniques?

Imputation fills missing values so models can use the full dataset. Common strategies: mean (for normally distributed numeric), median (for skewed numeric or when outliers exist), mode/most_frequent (for categorical or skewed numeric), KNN imputer (uses similar rows), and constant (fills with a fixed value). sklearn's SimpleImputer covers the first three.

Q3. When would you use mean vs. median vs. mode imputation?

Use mean when the feature is roughly normally distributed and has no significant outliers. Use median when the feature is skewed or has outliers (it is robust). Use mode (most_frequent) for categorical columns or highly discrete numerics.

Q4. What is feature scaling and why is it important?

Feature scaling brings numeric features to a comparable range so that distance-based (KNN, SVM) and gradient-based (logistic regression, neural networks) models are not dominated by features with large magnitudes. Without scaling, a feature with range 0–10,000 would overpower one with range 0–1.

Q5. What is StandardScaler and how does it work?

StandardScaler (z-score normalisation) subtracts the mean and divides by the standard deviation, producing features with mean=0 and std=1. It is appropriate when the feature roughly follows a normal distribution. Formula: $z = (x - \mu) / \sigma$.

Q6. What is MinMaxScaler and how does it differ from StandardScaler?

MinMaxScaler rescales values to a fixed range [0, 1] using (x – min)/(max – min). It is sensitive to outliers. StandardScaler is not bounded but is more robust to outliers. Use MinMaxScaler when you need a bounded output; use StandardScaler for normally distributed data.

Q7. What is RobustScaler and when should you use it?

RobustScaler uses the median and IQR instead of mean and std, making it robust to outliers. Use it when your numeric features have significant outliers that you do not want to remove but do not want to skew scaling.

Q8. What is One-Hot Encoding (OHE) and when is it used?

OHE creates a binary column for each category. E.g., 'Color' with [Red, Blue, Green] becomes three columns. Use OHE for nominal (unordered) categorical features. The handle_unknown='ignore' parameter prevents errors on unseen categories at inference time.

Q9. What is Label Encoding and when should you use it?

Label Encoding assigns an integer to each category (e.g., Red=0, Blue=1, Green=2). It is appropriate only for ordinal features (natural order exists) or tree-based models that are not affected by magnitude. For nominal features in linear models, OHE is preferred because Label Encoding implies a false ordinal

relationship.

Q10. What is Ordinal Encoding and how does it differ from Label Encoding?

Ordinal Encoding also maps categories to integers but allows you to specify the order explicitly (e.g., Low=0, Medium=1, High=2). Label Encoding assigns arbitrary integers. Use Ordinal Encoding when the category order matters.

Q11. What is Target Encoding?

Target Encoding replaces each category with the mean of the target variable for that category. It is compact and can be powerful for high-cardinality features but risks data leakage if not done carefully (use cross-validated target encoding on training data only).

Q12. What are outliers and how do you detect and handle them?

Outliers are data points far from the bulk of the distribution. Detection methods: IQR method (flag values beyond Q1 – 1.5×IQR or Q3 + 1.5×IQR), Z-score (|z| > 3), and visual inspection (box plots, scatter plots). Handling: remove them (if erroneous), cap/clip (Winsorising), transform (log), use robust models (tree-based), or use RobustScaler.

Q13. What is a Pipeline in sklearn and why use it?

A Pipeline chains preprocessing steps and a model into a single estimator. It ensures the same transformations are applied consistently to training and test data, prevents data leakage (fit only on train), makes code cleaner, and enables hyperparameter tuning of the entire workflow with GridSearchCV.

Q14. What is a ColumnTransformer?

ColumnTransformer applies different transformations to different columns simultaneously and concatenates the results. E.g., apply StandardScaler to numeric columns and OHE to categorical columns in one step before passing to a model.

Q15. What is the difference between fit(), transform(), and fit_transform()?

fit() learns parameters (e.g., mean, std) from training data. transform() applies the learned transformation. fit_transform() does both in one step. IMPORTANT: always fit only on training data, then transform both train and test to prevent data leakage.

Q16. What is data leakage?

Data leakage occurs when information from outside the training set (test/future data) influences the model during training, causing overly optimistic evaluation. Common causes: fitting scalers/encoders on the full dataset before splitting, or including target-derived features computed on the test set.

Q17. What is SMOTE and when is it used?

SMOTE (Synthetic Minority Oversampling Technique) generates synthetic samples for the minority class by interpolating between existing minority samples. It is used to handle class imbalance. Apply SMOTE only on training data to avoid leakage.

Q18. What are methods to handle class imbalance?

1) Oversampling minority class (SMOTE, random oversampling). 2) Undersampling majority class (random undersampling, Tomek links, cluster centroids). 3) Class weights: set class_weight='balanced' in sklearn models. 4) Use appropriate metrics (F1, AUC-ROC) instead of accuracy. 5) Ensemble methods like BalancedRandomForest.

FEATURE ENGINEERING & SELECTION

Q1. What is feature engineering?

Feature engineering is creating new informative features from existing ones to improve model performance. Examples: extracting hour/day from a datetime, combining two features (ratio, product), binning continuous values, extracting text statistics (word count, uppercase ratio), TF-IDF vectorisation.

Q2. What is dimensionality reduction? Does it discard data points?

Dimensionality reduction reduces the number of features while retaining as much information as possible. It does NOT discard data points (rows); it reduces columns. Methods: PCA (unsupervised, linear), LDA (supervised), t-SNE/UMAP (non-linear, for visualisation), and feature selection (RFE, SelectKBest).

Q3. What is PCA and how does it work?

Principal Component Analysis is an unsupervised linear dimensionality reduction technique. It finds orthogonal directions (principal components) of maximum variance in the data using eigenvectors of the covariance matrix. Each component is a linear combination of original features. PCA is linear; use Kernel PCA for non-linear data.

Q4. What are eigenvectors and how do you calculate PCA manually?

Eigenvectors of the covariance matrix define the principal components (directions of maximum variance). Steps: 1) Standardise data. 2) Compute covariance matrix. 3) Compute eigenvalues and eigenvectors. 4) Sort eigenvectors by descending eigenvalue. 5) Project data onto top-k eigenvectors. PCA is based on SVD (Singular Value Decomposition).

Q5. What is RFE (Recursive Feature Elimination)?

RFE recursively trains a model, ranks features by importance, and removes the least important feature(s) in each iteration until the desired number remains. It requires a model that provides feature importances or coefficients (e.g., LinearSVC, RandomForestClassifier). sklearn: from sklearn.feature_selection import RFE.

Q6. What is the difference between parameters and hyperparameters?

Parameters are learned from the data during training (e.g., weights in logistic regression, split thresholds in decision trees). Hyperparameters are set before training and control the learning process (e.g., n_estimators, max_depth, learning_rate, C, kernel). You tune hyperparameters; the model learns parameters.

HYPERPARAMETER TUNING

Q1. What is hyperparameter tuning and why is it needed?

HPT finds the optimal set of hyperparameters that maximise model performance on validation data. Without tuning, you rely on default values that may not suit your data. Tuning prevents underfitting (model too simple) or overfitting (model too complex).

Q2. What is GridSearchCV?

GridSearchCV exhaustively tries every combination of the provided hyperparameter grid with cross-validation, selecting the combination with the best CV score. It is thorough but computationally expensive (fits = n_combinations × cv_folds).

Q3. What is RandomizedSearchCV and how does it differ from GridSearchCV?

RandomizedSearchCV samples a fixed number (n_iter) of hyperparameter combinations at random. It is faster than GridSearchCV and often finds near-optimal results when the grid is large. Trade-off: it may miss the global optimum. Number of fits = n_iter × cv_folds.

Q4. What is cross-validation (CV) and why is it used?

Cross-validation splits training data into k folds, trains on k−1 folds, and validates on the remaining fold, rotating k times. It gives a more reliable performance estimate than a single train/validation split and reduces variance due to the split. The cv parameter in GridSearchCV/RandomizedSearchCV controls this.

Q5. How many total fits happen in GridSearchCV vs RandomizedSearchCV?

GridSearchCV: total fits = (number of all parameter combinations) × cv. RandomizedSearchCV: total fits = n_iter × cv. For example, a grid of 3×3×2=18 combinations with cv=5 → 90 fits in GridSearch; with n_iter=10, cv=5 → 50 fits in RandomizedSearch.

Q6. What is learning rate and how does it affect model training?

Learning rate (η) controls the step size when updating model weights during gradient descent. A high learning rate converges fast but may overshoot the minimum (diverge or oscillate). A low learning rate is stable but slow to converge and may get stuck in local minima. Typical values: 0.01–0.1.

Q7. What is random_state=42 and why 42?

random_state seeds the random number generator so results are reproducible (same splits, same initialisation). 42 is a cultural reference to 'The Hitchhiker's Guide to the Galaxy' where 42 is 'the answer to everything.' Any integer works; 42 is just convention. Saying only the pop-culture answer without explaining reproducibility is incomplete.

MODEL EVALUATION METRICS

Q1. What is a confusion matrix?

A confusion matrix is a table comparing actual vs. predicted class labels. For binary classification: TP (correctly predicted positive), TN (correctly predicted negative), FP (actual negative predicted positive), FN (actual positive predicted negative). For multi-class, it is an N×N matrix where rows = actual, columns = predicted.

Q2. What is Precision, Recall, and F1-score?

Precision = TP / (TP + FP) — of all predicted positives, how many are actually positive. Recall (Sensitivity) = TP / (TP + FN) — of all actual positives, how many did we catch. F1 = 2 × (Precision × Recall) / (Precision + Recall) — harmonic mean balancing both. Use Precision when false positives are costly; Recall when false negatives are costly (e.g., disease detection).

Q3. What is macro vs. weighted vs. micro average in classification report?

Macro average: compute metric for each class independently, then take unweighted mean — treats all classes equally. Weighted average: mean weighted by support (number of true instances per class) — accounts for class imbalance. Micro average: aggregate all TP, FP, FN globally before computing.

Q4. What is 'support' in a classification report?

Support is the number of actual occurrences of each class in the test set. It tells you how many samples each metric is based on, helping identify if a class has very few test samples (making its metrics less reliable).

Q5. What is AUC-ROC and how do you interpret it?

ROC (Receiver Operating Characteristic) curve plots True Positive Rate vs. False Positive Rate at various thresholds. AUC (Area Under Curve) summarises this in a single number: 0.5 = random classifier, 1.0 = perfect. Higher AUC = better ability to distinguish classes. Useful for imbalanced datasets or when you care about ranking predictions.

Q6. Why is accuracy not always a good metric?

Accuracy is misleading when classes are imbalanced. E.g., a dataset with 95% negative samples: a model predicting all negatives gets 95% accuracy but detects no positives. F1-score, precision-recall, and AUC-ROC are better choices for imbalanced data.

Q7. What is R2-score (coefficient of determination)?

R2 measures how well a regression model explains the variance in the target. R2 = 1 – (SS_residual / SS_total). R2=1 is a perfect fit; R2=0 means the model is no better than predicting the mean; R2 can be negative if the model is worse than the mean. It is scale-independent, unlike RMSE.

Q8. What is the difference between RMSE and R2?

RMSE (Root Mean Squared Error) = sqrt(mean((y_pred – y_true)²)). It is in the same unit as the target and penalises large errors more. R2 is unitless and measures the proportion of variance explained. RMSE is better for comparing model errors in absolute terms; R2 is better for explaining goodness of fit.

Q9. What is MSE and MAE?

MSE = mean((y_pred – y_true)²) — penalises large errors quadratically. MAE = mean(|y_pred – y_true|) — equally penalises all errors. MSE is differentiable everywhere (good for gradient-based methods); MAE is more robust to outliers.

MACHINE LEARNING ALGORITHMS

Q1. How does Logistic Regression work?

Logistic Regression applies the sigmoid function (σ(z) = 1/(1+e^–z)) to a linear combination of features to output a probability between 0 and 1. A threshold (usually 0.5) converts it to a class. It is a discriminative linear model. Despite the name 'regression,' it is used for classification.

Q2. Why is Logistic Regression called 'Regression' if it is used for classification?

Because it models the log-odds (logit) of the probability as a linear regression: log(p/(1–p)) = w■x + b. The regression part refers to estimating a continuous probability, which is then thresholded for classification.

Q3. What is the loss function of Logistic Regression?

Binary Cross-Entropy (Log Loss): L = –[y·log(p) + (1–y)·log(1–p)]. It penalises confident wrong predictions heavily. MSE is not used because it creates a non-convex loss surface for logistic regression, making optimisation harder.

Q4. What is the 'C' parameter in Logistic Regression / SVM?

C is the inverse of regularisation strength. A large C means less regularisation (model tries hard to classify all training points correctly, risks overfitting). A small C means more regularisation (allows more misclassifications, simpler model, may underfit). So C is the trade-off between training accuracy and generalisation.

Q5. What is the 'solver' parameter in Logistic Regression?

The solver specifies the optimisation algorithm: 'lbfgs' (default, good for multiclass, memory-efficient quasi-Newton), 'saga' (supports all penalties including L1, good for large datasets), 'liblinear' (good for small datasets, only binary+OVR), 'newton-cg' / 'sag' (for L2 only).

Q6. Is Logistic Regression a parametric or non-parametric model?

Parametric — it assumes a fixed functional form (linear relationship between features and log-odds) and learns a fixed number of parameters (weights). The model complexity does not grow with training data size.

Q7. How does a Decision Tree work and how is the root node selected?

A Decision Tree recursively splits data on the feature and threshold that maximises information gain (or minimises impurity). For classification, impurity is measured by Gini index or Entropy. The feature with the highest information gain (= parent impurity – weighted child impurity) becomes the root. A pure node (all one class) has Gini=0 / Entropy=0 and is not split further.

Q8. What is entropy and information gain?

Entropy H = –Σ p_i·log2(p_i) measures disorder/uncertainty. A pure node (all one class) has H=0; maximum disorder has H=1 for binary. Information Gain = Parent entropy – weighted average of child entropies after a split. Higher IG = better split.

Q9. What is the Gini index and its range?

Gini = 1 – Σ p_i² for each class i. Range: 0 (pure node) to 0.5 (binary, maximum impurity). The Decision Tree picks the split that minimises the weighted Gini of child nodes. Gini is faster to compute than entropy.

Q10. What is the difference between a pure node and an impure node?

A pure node contains samples of only one class (Gini=0, Entropy=0) — no further splitting is needed. An impure node contains a mix of classes; splitting it can reduce impurity and improve predictions.

Q11. What is Random Forest and how does it work?

Random Forest is a bagging ensemble of Decision Trees. Each tree is trained on a bootstrap sample (random sample with replacement) of the training data. At each split, only a random subset of features (√n_features for classification) is considered. The final prediction is the majority vote (classification) or average (regression). This reduces variance vs. a single deep tree.

Q12. Is Random Forest parametric or non-parametric?

Non-parametric — it makes no assumptions about the functional form of the data and complexity grows with data. The individual Decision Trees store split rules rather than learned weight vectors.

Q13. What are the parameters of Random Forest and what do they do?

n_estimators: number of trees (more = more stable but slower). max_depth: maximum depth of each tree (limits complexity, prevents overfitting). min_samples_split: min samples to split a node. min_samples_leaf: min samples in a leaf. max_features: features considered per split. bootstrap: whether to use bootstrap samples.

Q14. What is bagging and boosting?

Bagging (Bootstrap AGGregating): trains multiple models in PARALLEL on random subsets of data, then aggregates predictions (reduces variance). Example: Random Forest. Boosting: trains models SEQUENTIALLY, each correcting the errors of the previous (reduces bias). Examples: AdaBoost, Gradient Boosting, XGBoost, LightGBM.

Q15. Which reduces variance and which reduces bias — bagging or boosting?

Bagging reduces variance (models are independent, averaging cancels noise). Boosting reduces bias (sequential focus on difficult examples). Bagging is better when the model overfits; boosting is better when the model underfits.

Q16. How does XGBoost work?

XGBoost (Extreme Gradient Boosting) is a regularised, optimised gradient boosting framework. It builds trees sequentially, where each tree fits the negative gradient (residuals) of the loss function. It adds L1/L2 regularisation, handles missing values internally, uses histogram-based splits, and supports parallel tree construction. Key hyperparameters: n_estimators, learning_rate, max_depth, subsample, colsample_bytree.

Q17. What is the difference between XGBoost and LightGBM?

LightGBM uses a leaf-wise (best-first) tree growth strategy while XGBoost uses level-wise. Leaf-wise grows the leaf with the highest loss reduction, leading to deeper trees but faster convergence. LightGBM also uses histogram-based feature binning, making it faster and more memory-efficient on large datasets. XGBoost can be slower but sometimes more accurate on smaller datasets.

Q18. What does 'Light' in LightGBM mean?

Light refers to the lightweight, memory-efficient design: histogram-based feature binning reduces the data into discrete bins, cutting memory and compute. It supports GPU training and can handle very large datasets efficiently.

Q19. What is Gradient Boosting and how does it relate to XGBoost?

Gradient Boosting iteratively adds weak learners (usually small trees) that fit the negative gradient of the loss function (residuals). XGBoost is an implementation of gradient boosting with additional regularisation (L1/L2), efficient histogram-based splits, and parallelisation.

Q20. What is Gradient Descent?

Gradient Descent is an iterative optimisation algorithm that minimises a loss function by moving in the direction of the negative gradient. Update rule: $w \leftarrow w - \eta \cdot \nabla L(w)$, where η is the learning rate. Variants: Batch GD (all data), Stochastic GD (one sample), Mini-batch GD (subset).

Q21. How does KNN (K-Nearest Neighbours) work?

KNN is a lazy, non-parametric algorithm. For a new point, it finds the k nearest training points (by Euclidean/other distance), then predicts the majority class (classification) or average (regression) among those k neighbours. Small k → low bias, high variance (overfitting); large k → high bias, low

variance (underfitting). KNN is sensitive to feature scaling.

Q22. Why is KNN called a lazy learner?

Because it does not build an explicit model during training — it simply stores all training data. All computation happens at prediction time (searching for nearest neighbours). This makes training O(1) but prediction O(n) for n training points.

Q23. How does SVM (Support Vector Machine) work?

SVM finds the hyperplane that maximises the margin between classes (distance between the hyperplane and the nearest training points of each class, called support vectors). Only support vectors determine the hyperplane — other points don't matter. For non-linearly separable data, the kernel trick maps inputs to higher dimensions. C controls regularisation (slack variables).

Q24. What are kernel types in SVM?

Linear: for linearly separable data. RBF (Radial Basis Function): maps to infinite dimensions, good for non-linear data. Polynomial: computes polynomial combinations of features. Sigmoid: similar to neural network activation. RBF is the most commonly used default.

Q25. What is One-vs-One and One-vs-All (OvR) in multi-class SVM?

One-vs-All (OvR): train one binary classifier per class (class k vs. all others). Predict the class whose classifier gives the highest confidence. One-vs-One (OvO): train a classifier for every pair of classes (n(n-1)/2 classifiers), predict by majority vote. OvO is typically more accurate; OvR is faster to train.

Q26. What is L1 and L2 regularisation?

L1 (Lasso): adds Σ|w■| penalty. Produces sparse models (some weights go exactly to 0), useful for feature selection. L2 (Ridge): adds Σw■² penalty. Shrinks all weights towards 0 but rarely eliminates them, preventing overfitting while keeping all features. ElasticNet combines both.

Q27. When to use L1 vs. L2?

Use L1 when you suspect many features are irrelevant (built-in feature selection). Use L2 when all features are potentially relevant but you want to prevent large weights (most common default). ElasticNet is a safe middle ground.

OVERFITTING & UNDERFITTING

Q1. What is overfitting and underfitting?

Overfitting: model learns training data too well including noise, so it performs well on train but poorly on unseen test data (high variance, low bias). Underfitting: model is too simple to capture patterns, performing poorly on both train and test (high bias, low variance).

Q2. What is the bias-variance tradeoff?

Bias: error due to overly simplistic assumptions in the learning algorithm (underfitting). Variance: error due to sensitivity to fluctuations in training data (overfitting). Increasing model complexity decreases bias but increases variance. The goal is to find a balance that minimises total error on test data.

Q3. How do you detect if a model is overfitting or underfitting?

Compare training and validation/test scores. Overfitting: train score >> validation score (large gap). Underfitting: both train and validation scores are low. Ideal: train and validation scores are both high and close to each other.

Q4. How do you reduce overfitting?

1) Get more training data. 2) Reduce model complexity (fewer layers/trees/depth). 3) Regularisation (L1/L2/dropout). 4) Cross-validation. 5) Early stopping. 6) Pruning (for decision trees). 7) Bagging/ensemble methods. 8) Feature selection (remove irrelevant features). 9) Data augmentation.

Q5. How do you reduce underfitting?

1) Increase model complexity (more features, deeper trees, more neurons). 2) Add more relevant features. 3) Reduce regularisation. 4) Train longer. 5) Use a more powerful algorithm. 6) Feature engineering.

Q6. If a model is overfitting and you use XGBoost, should you increase or decrease n_estimators?

Decrease n_estimators (fewer trees reduces model complexity and overfitting). Also decrease learning_rate, max_depth, and increase min_child_weight/lambda. Conversely, if underfitting, increase n_estimators.

NLP & TEXT FEATURES

Q1. What is TF-IDF and how does it work?

TF-IDF (Term Frequency–Inverse Document Frequency) converts text to numeric vectors. TF(t,d) = (count of term t in document d) / (total terms in d). IDF(t) = log(N / df(t)), where N = total documents, df(t) = documents containing t. TF-IDF = TF × IDF. Common words (high df) get low IDF; rare informative words get high IDF.

Q2. What are the key parameters of TfidfVectorizer?

max_features: vocabulary size limit. ngram_range: e.g., (1,2) includes unigrams and bigrams. min_df: ignore terms in fewer than min_df documents (reduces noise). max_df: ignore terms in more than max_df fraction (removes stop words automatically). stop_words: built-in or custom stop word list. sublinear_tf: apply log to TF.

Q3. What are n-grams (unigrams, bigrams, trigrams)?

An n-gram is a contiguous sequence of n words. Unigram (n=1): single words. Bigram (n=2): two consecutive words (captures context). Trigram (n=3): three consecutive words. Larger n captures more context but increases sparsity and computation.

Q4. What is tokenisation?

Tokenisation splits text into tokens (words, subwords, or characters). It is the first step in NLP: convert a string into a list of tokens that can then be numerically represented.

Q5. What are stop words?

Stop words are common words (e.g., 'the', 'is', 'and') that appear frequently but carry little meaning for most NLP tasks. Removing them reduces vocabulary size and noise. However, for some tasks (sentiment, negation detection) stop words may be important and should be kept.

CODING QUESTIONS

Q1. How do you load a standard sklearn dataset?

```
from sklearn.datasets import load_iris, load_breast_cancer, load_digits, load_wine
iris = load_iris() # or load_digits() etc.
X, y = iris.data, iris.target
print(X.shape, y.shape)
```

Q2. How do you perform train-test split?

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)
print(X_train.shape, X_test.shape)
```

Q3. How do you build a basic sklearn Pipeline?

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
('imputer', SimpleImputer(strategy='mean')),
('scaler', StandardScaler()),
('model', LogisticRegression())
])
pipe.fit(X_train, y_train)
print(pipe.score(X_test, y_test))
```

Q4. How do you build a ColumnTransformer for mixed data?

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline

num_pipe = Pipeline([('imputer', SimpleImputer(strategy='median')), ('scaler',
StandardScaler())])
cat_pipe = Pipeline([('imputer', SimpleImputer(strategy='most_frequent')), ('encoder',
OneHotEncoder(handle_unknown='ignore'))])

preprocessor = ColumnTransformer([
('num', num_pipe, num_cols),
('cat', cat_pipe, cat_cols)
])
```

Q5. How do you use SimpleImputer standalone?

```
from sklearn.impute import SimpleImputer
import numpy as np

imp = SimpleImputer(strategy='mean') # or 'median', 'most_frequent', 'constant'
imp.fit(X_train[['col']])
X_train[['col']] = imp.transform(X_train[['col']])
X_test[['col']] = imp.transform(X_test[['col']])
```

Q6. How do you apply RFE to select top 2 features?

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)
rfe = RFE(estimator=model, n_features_to_select=2)
rfe.fit(X_train, y_train)
print('Selected features:', X.columns[rfe.support_].tolist())
```

Q7. How do you print column names and find null values?

```
print(df.columns.tolist()) # all column names
print(df.dtypes) # data types
print(df.isnull().sum()) # null count per column
print(df.shape) # rows, columns
```

Q8. How do you separate numerical and categorical columns?

```
num_cols = df.select_dtypes(include='number').columns.tolist()
cat_cols = df.select_dtypes(include='object').columns.tolist()
print('Numeric:', num_cols)
print('Categorical:', cat_cols)
```

Q9. How do you print value counts / frequency of a column?

```
print(df['column'].value_counts()) # frequency of each unique value
print(df['column'].unique()) # unique values
print(df['column'].nunique()) # count of unique values
```

Q10. How do you apply RandomizedSearchCV?

```
from sklearn.model_selection import RandomizedSearchCV

param_dist = {'n_estimators': [100, 200, 300], 'max_depth': [3, 5, 10, None]}
rs = RandomizedSearchCV(estimator, param_dist, n_iter=10, cv=5, scoring='f1_macro',
random_state=42)
rs.fit(X_train, y_train)
print(rs.best_params_, rs.best_score_)
```

Q11. How do you print a classification report?

```
from sklearn.metrics import classification_report, confusion_matrix
y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

Q12. How do you change the strategy in SimpleImputer from median to mode?

Change strategy='median' to strategy='most_frequent'. Note: 'most_frequent' is the exact string for mode imputation in sklearn.

Q13. How do you replace missing/unknown values in a column?

```
df['col'] = df['col'].replace({'?': np.nan, 'unknown': np.nan, 'UNKNOWN': 'Other'})
df['col'].fillna(df['col'].mode()[0], inplace=True)
```

ADVANCED THEORY

Q1. What are the types of Machine Learning?

Supervised Learning: model learns from labelled data (classification, regression). Unsupervised Learning: finds patterns in unlabelled data (clustering, dimensionality reduction). Semi-supervised: uses a small amount of labelled + large unlabelled data. Reinforcement Learning: agent learns by interacting with an environment and receiving rewards/penalties.

Q2. What is the difference between Machine Learning and Deep Learning?

ML algorithms typically require manual feature engineering and work well on structured tabular data. Deep Learning uses multi-layer neural networks that automatically learn hierarchical features from raw data (images, text, audio). DL needs much more data and compute but often outperforms traditional ML on unstructured data.

Q3. What are activation functions? Name common ones.

Activation functions introduce non-linearity into neural networks, enabling them to learn complex patterns. Common ones: ReLU f(x) = max(0, x) — most common, avoids vanishing gradient. Sigmoid σ(x) = 1/(1+e^–x) — outputs [0,1], used in output layer for binary classification. Tanh: outputs [–1,1], stronger than sigmoid. Softmax: converts logits to probabilities for multi-class output.

Q4. What is the input and output of ReLU?

ReLU takes any real number as input. Output: max(0, x). Negative inputs → 0; positive inputs → same value. It is non-saturating for positive values (no vanishing gradient issue) but can 'die' (always output 0) for negative inputs.

Q5. What is the input and output of Tanh?

Tanh takes any real number as input. Output range: (–1, 1). It is zero-centred (unlike sigmoid), which can speed up convergence. However, it still suffers from the vanishing gradient problem for very large or small inputs.

Q6. What is a loss function?

A loss function measures how far the model's predictions are from the true values. The model is trained by minimising this function via gradient descent. For classification: Cross-Entropy / Log Loss. For regression: MSE, MAE, Huber Loss. The loss function must be differentiable for gradient-based optimisation.

Q7. What is K-Means clustering and how does it work?

K-Means partitions data into k clusters. Steps: 1) Randomly initialise k centroids. 2) Assign each point to the nearest centroid. 3) Recompute centroids as the mean of assigned points. 4) Repeat 2–3 until centroids converge. Limitation: requires specifying k, sensitive to initialisation and outliers.

Q8. What is the elbow method for finding optimal k in K-Means?

Plot inertia (sum of squared distances to nearest centroid) vs. number of clusters k. The 'elbow' — where the rate of decrease sharply slows — suggests the optimal k. Beyond the elbow, adding more clusters gives diminishing returns.

Q9. What is oversampling and when does it cause problems?

Oversampling increases the minority class representation. Random oversampling duplicates minority samples (risks exact duplicate noise). SMOTE creates synthetic samples. Problem: if applied before train/test split, synthetic minority test samples leak training information → overly optimistic evaluation. Always apply oversampling only within the training fold.

Q10. What is the difference between a tree and a graph?

A tree is a connected, acyclic graph with a hierarchy (root, parent, children, leaves). A graph is a general structure of nodes and edges that may have cycles and no fixed root. Decision trees are trees; neural network architectures (with skip connections) or knowledge graphs are graphs.

Q11. What is the height vs. depth of a decision tree?

Depth: the length of the longest path from the root to a leaf (how many levels). Height of a node: the length of the longest path from that node to a leaf. For a binary tree with depth d, the maximum number of nodes = 2^(d+1) – 1.